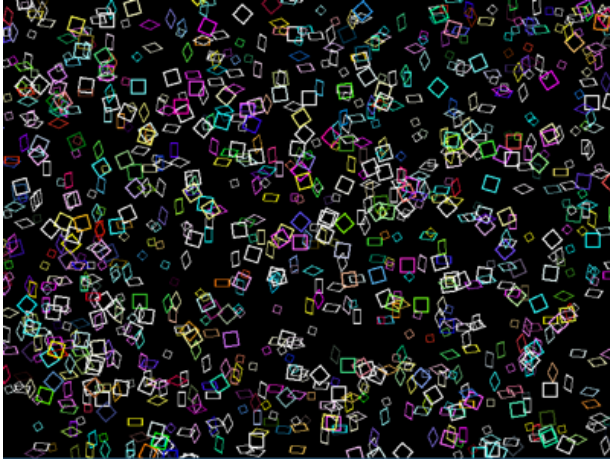


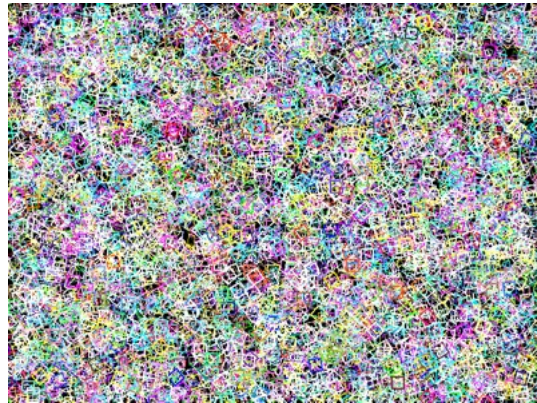
R1.01

CM 9 : Introduction à la SAE

Projet particule



Un système de particules gère en ensemble de particules en mettant à jour leurs caractéristiques de manière régulière au cours du temps.



Projet Particules

Une particule est un objet possédant :

- un certain nombre de caractéristiques permettant de l'afficher à l'écran : au minimum une position, mais éventuellement aussi un angle de rotation, une taille (ou une échelle par rapport à la taille d'une particule de base), une couleur, une transparence, etc.
- un certain nombre de caractéristiques permettant de faire évoluer son état au cours du temps : vitesse, accélération, durée de vie, fonctions d'évolution de la couleur ou de la transparence, capacité d'interaction avec les autres particules, etc.

Un système de particules gère un ensemble de particules en :

- engendrant de nouvelles particules au fil du temps en fonction de ses caractéristiques (taux de génération, position du générateur, etc),
- mettant à jour ses particules (en particulier leur position) au fil du temps en fonction des caractéristiques de celles-ci (en particulier leur vitesse),
- supprimant éventuellement les particules dont la durée de vie est épuisée.

Mise en œuvre

- Groupes de 2 du même TP, un groupe de 3 est possible par TP
- Possibilité de note différenciée
- 2 Rendus :
 - Semaine 50, un système de base
 - Semaine 3, évaluation par les pairs
 - Semaine 4, rendu et évaluation orale

Attentes :

- Algorithmique
 - Un ou des problèmes algorithmiques, par exemple optimiser la détection de la collision entre particules, optimiser la gestion des particules qui quittent l'écran, ...
 - Performance en temps et en espace
 - Lisibilité / documentation
- Programmation
 - Organisation du code
 - Visibilité, package
 - Tests
- Projets
 - Rendu
 - Sens
 - Usage

Organisation des sources du projet

Fichier go à la racine, à ne pas modifier sauf si justification et validation par l'enseignant.

`main.go` : le programme principale, charge la configuration, crée le système de particule et lance le jeux

`draw.go` : affiche l'état du système de particule (peu être amenée à être modifiée)

`game.go` : un type qui contient le système de particules

`layout.go` : permet de définir la taille logique de l'écran

`update.go` : met à jour le système

Fichiers go à la racine – game.go

```
package main
```

```
import "project-particles/particles"
```

/ game est un type qui implémente l'interface Game de la bibliothèque Ebiten car il dispose d'une méthode Update, d'une méthode Draw et d'une méthode Layout. C'est un élément de ce type qui est utilisé pour mettre à jour et afficher un système de particules. Ce type ne devrait pas être modifié sauf pour les deux dernières extensions. */*

```
type game struct {  
    system particles.System  
}
```

Fichiers go à la racine – main.go

```
package main
```

```
import (  
    "github.com/hajimehoshi/ebiten/v2"  
    "log"  
    "project-particles/assets"  
    "project-particles/config"  
    "project-particles/particles"  
)
```

/ main est la fonction principale du projet. Elle commence par lire le fichier de configuration, puis elle charge en mémoire l'image d'une particule. Elle initialise ensuite la fenêtre d'affichage, puis elle crée un système de particules encapsulé dans un "game" et appelle la fonction RunGame qui se charge de faire les mise-à-jour (Update) et affichages (Draw) de manière régulière*/*

```
func main() {  
    config.Get("config.json")  
    assets.Get()  
  
    ebiten.SetWindowTitle(config.General.WindowTitle)  
    ebiten.SetWindowSize(config.General.WindowSizeX, config.General.WindowSizeY)  
  
    g := game{system: particles.NewSystem()}  
  
    err := ebiten.RunGame(&g)  
    if err != nil {  
        log.Print(err)  
    }  
}
```

Fichiers go à la racine - layout

- package main

```
// Layout définit la taille en pixels de la zone d'affichage des particules  
// en fonction de la taille en pixels de la fenêtre. Vous n'avez jamais à  
// modifier cette fonction.  
func (g *game) Layout(outsideWidth, outsideHeight int) (int, int) {  
    return outsideWidth, outsideHeight  
}
```

package main

```
// Layout définit la taille en pixels de la zone d'affichage des particules  
// en fonction de la taille en pixels de la fenêtre. Vous n'avez jamais à  
// modifier cette fonction.  
func (g *game) Layout(outsideWidth, outsideHeight int) (int, int) {  
    return outsideWidth, outsideHeight  
}
```


Fichiers go à la racine - update

```
package main
```

```
// Update se charge d'appeler la fonction Update du système de particules  
// g.system. Elle est appelée automatiquement exactement 60 fois par  
seconde par  
// la bibliothèque Ebiten. Cette fonction ne devrait pas être modifiée sauf  
// pour les deux dernières extensions.
```

```
func (g *game) Update() error {
```

```
    g.system.Update()
```

```
    return nil
```

```
}
```

Fichiers go à la racine - update

```
package main

import ( ... )

// Draw se charge d'afficher à l'écran l'état actuel du système de particules
// g.system. Elle est appelée automatiquement environ 60 fois par seconde par
// la bibliothèque Ebiten. Cette fonction pourra être légèrement modifiée quand
// c'est précisé dans le sujet.
func (g *game) Draw(screen *ebiten.Image) {
    for e := g.system.Content.Front(); e != nil; e = e.Next() {
        p, ok := e.Value.(*particles.Particle)
        if ok {
            options := ebiten.DrawImageOptions{}
            options.GeoM.Rotate(p.Rotation)
            options.GeoM.Scale(p.ScaleX, p.ScaleY)
            options.GeoM.Translate(p.PositionX, p.PositionY)
            options.ColorM.Scale(p.ColorRed, p.ColorGreen, p.ColorBlue, p.Opacity)
            screen.DrawImage(assets.ParticleImage, &options)
        }
    }

    if config.General.Debug {
        ebitenutil.DebugPrint(screen, fmt.Sprintf(ebiten.CurrentTPS()))
    }
}
```

Fichiers go à la racine – config.json

```
{
  "WindowTitle": "Project
particles",
  "WindowSizeX": 800,
  "WindowSizeY": 600,
  "ParticleImage":
"assets/particle.png",
  "Debug": true,
  "InitNumParticles": 10,
  "RandomSpawn": true,
  "SpawnX": 400,
  "SpawnY": 400,
  "SpawnRate": 1
}
```

- JSON :JavaScript Object Notation :
un format pour représenter un
fichier JavaScript
- Utilisé pour définir la configuration
de l'applications géré par
config/Get qui lit le fichier est
stocke la valeur dans
config/General avec le type
config/Config

Package assets – charge une particule en mémoire

```
package assets
```

```
import (  
    "github.com/hajimehoshi/ebiten/v2"  
    "github.com/hajimehoshi/ebiten/v2/ebitenutil"  
    "image/png"  
    "log"  
    "project-particles/config"  
)
```

```
// ParticleImage est une variable globale pour stocker l'image d'une particule  
var ParticleImage *ebiten.Image
```

```
/* Get charge en mémoire l'image de la particule. (particle.png)  
   Vous pouvez changer cette image si vous le souhaitez, et même en proposer plusieurs, qu'on peut choisir via le fichier de  
   configuration. Cependant ceci n'est pas du tout central dans le projet et ne devrait être fait que  
   si vous avez déjà bien avancé sur tout le reste.*/
```

```
func Get() {  
    var err error  
    ParticleImage, _, err = ebitenutil.NewImageFromFile(config.General.ParticleImage)  
    if err != nil {  
        log.Fatal("Problem while loading particle image: ", err)  
    }  
}
```

Package config – charge la configuration définie par config.json

package config

```
// Config définit les champs qu'on peut trouver dans un fichier de config.  
// Dans le fichier les champs doivent porter le même nom que dans le type si  
// dessous, y compris les majuscules. Tous les champs doivent obligatoirement  
// commencer par des majuscules, sinon il ne sera pas possible de récupérer  
// leurs valeurs depuis le fichier de config.  
// Vous pouvez ajouter des champs et ils seront automatiquement lus dans le  
// fichier de config. Vous devrez le faire plusieurs fois durant le projet.
```

```
type Config struct {  
    WindowTitle      string  
    WindowSizeX, WindowSizeY int  
    ParticleImage     string  
    Debug             bool  
    InitNumParticles  int  
    RandomSpawn       bool  
    SpawnX, SpawnY    int  
    SpawnRate         float64  
}
```

```
var General Config
```

package config

```
import (  
    "encoding/json"  
    "log"  
    "os"  
)
```

```
// Get récupère le contenu du fichier config.json et le stocke dans la variable  
// General du package config. Normalement vous ne devriez jamais modifier cette  
// fonction.
```

```
func Get(fileName string) {  
  
    content, err := os.ReadFile(fileName)  
    if err != nil {  
        log.Fatal("Error when opening config file: ", err)  
    }  
  
    err = json.Unmarshal(content, &General)  
    if err != nil {  
        log.Fatal("Error when reading config file: ", err)  
    }  
}
```

Une structure de donnée fournie avec go

<https://pkg.go.dev/container/list>

Listes doublement chaînées

- type Element
 - func (e *Element) Next() *Element
 - func (e *Element) Prev() *Element
- type List
 - func New() *List
 - func (l *List) Back() *Element
 - func (l *List) Front() *Element
 - func (l *List) Init() *List
 - func (l *List) InsertAfter(v any, mark *Element) *Element
 - func (l *List) InsertBefore(v any, mark *Element) *Element
 - func (l *List) Len() int
 - func (l *List) MoveAfter(e, mark *Element)
 - func (l *List) MoveBefore(e, mark *Element)
 - func (l *List) MoveToBack(e *Element)
 - func (l *List) MoveToFront(e *Element)
 - func (l *List) PushBack(v any) *Element
 - func (l *List) PushBackList(other *List)
 - func (l *List) PushFront(v any) *Element
 - func (l *List) PushFrontList(other *List)
 - func (l *List) Remove(e *Element) any

Package particules - types

```
package particles
```

```
import "container/list"
```

```
// System définit un système de particules.  
// Pour le moment il ne contient qu'une liste de particules, mais cela peut  
// évoluer durant votre projet.
```

```
type System struct {  
    Content *list.List  
}
```

```
// Particle définit une particule.  
// Elle possède une position, une rotation, une taille, une couleur, et une  
// opacité. Vous ajouterez certainement d'autres caractéristiques aux particules  
// durant le projet.
```

```
type Particle struct {  
    PositionX, PositionY float64  
    Rotation float64  
    ScaleX, ScaleY float64  
    ColorRed, ColorGreen, ColorBlue float64  
    Opacity float64  
}
```

Package particules - NewSystem

```
package particles
```

```
import (  
    "container/list"  
    "project-particles/config"  
)
```

```
// NewSystem est une fonction qui initialise un système de particules et le  
// retourne à la fonction principale du projet, qui se chargera de l'afficher.  
// C'est à vous de développer cette fonction.  
// Dans sa version actuelle, cette fonction affiche une particule blanche au  
// centre de l'écran.
```

```
func NewSystem() System {  
    l := list.New()  
    l.PushFront(&Particle{  
        PositionX: float64(config.General.WindowSizeX) / 2,  
        PositionY: float64(config.General.WindowSizeY) / 2,  
        ScaleX: 1, ScaleY: 1,  
        ColorRed: 1, ColorGreen: 1, ColorBlue: 1,  
        Opacity: 1,  
    })  
    return System{Content: l}  
}
```


Package particules - Update

`package` `particules`

*// Update mets à jour l'état du système de particules (c'est-à-dire
l'état de
// chacune des particules) à chaque pas de temps. Elle est appelée
exactement
// 60 fois par seconde (de manière régulière) par la fonction principale
du
// projet.
// C'est à vous de développer cette fonction.*
`func` (`s` `*System`) `Update()` {}

Attention

Toutes les fonctions devront-êtré associée à des tests

Les fonctions devront-êtré les plus élémentaires possibles

Système de particules de base : objectif

Coder un système de particules basique : chaque particule a une vitesse initiale (qui n'évolue pas au cours du temps), une position initiale, et sa position évolue au fil du temps en fonction de sa vitesse. Les autres caractéristiques des particules (taille, rotation, couleur, transparence) n'évoluent pas.

Système de particules de base - NewSystem()

Coder la fonction NewSystem() (fichier particles/new.go) afin qu'elle crée un certain nombre de particules en leur donnant à chacune une position initiale.

Les contraintes suivantes devront être respectées :

- le nombre de particules devra être InitNumParticles, qui est défini dans le fichier config.json (il faudra donc l'obtenir en lisant la variable General du paquet config)
- la position des particules devra être soit aléatoire soit la même pour toutes les particules, en fonction du champs RandomSpawn du fichier config.json (encore une fois, il faudra utiliser la variable General du paquet config pour obtenir cette valeur),
- si la position des particules est aléatoire il faudra utiliser le paquet math/rand pour la définir pour chaque particule, en s'assurant qu'elle reste toujours à l'intérieur de l'écran (la taille de l'écran se trouve dans la variable General du paquet config,
- si la position des particules n'est pas aléatoire elle devra être définie par les champs SpawnX et SpawnY du fichier config.json (encore une fois, il faudra utiliser la variable General du paquet config pour obtenir ces valeurs).

Système de particules de base - Update() : vitesse des particules

Pour animer les particules :

- la fonction Update() se contentera, pour chaque particule d'ajouter sa vitesse selon la coordonnée x à sa position en x et sa vitesse selon la coordonnée y à sa position en y,
- la fonction Update() est automatiquement appelée 60 fois par seconde, de manière régulière, par le programme principal (déjà codé) du projet.

Rem: NewSystem() devra être modifiée

- les vitesses seront définies selon deux coordonnées (x et y), on aura donc une vitesse pour chaque coordonnée,
- chaque particule aura ses propres vitesses, définies aléatoirement (utiliser math/rand) et une fois pour toutes par NewSystem(),

Système de particules de base - Update() : génération de particules

Enrichir votre fonction Update() pour engendrer de nouvelles particules au cours du temps.

Pour cela, on dispose d'un taux de génération (SpawnRate dans la config) qui indique combien de nouvelles particules doivent être engendrées à chaque appel à la fonction Update(). Il s'agit d'un nombre décimal (ainsi, s'il vaut 3 on doit engendrer 3 particules lors de chaque appel à Update() et s'il vaut 0.5 on doit engendrer une nouvelle particule lors d'un appel à Update() sur deux).

Les nouvelles particules seront engendrées comme les particules de départ (position aléatoire ou la même pour toutes, selon la configuration, vitesse aléatoire), et elles seront bien sûr mises-à-jour comme les autres par Update() pour faire évoluer leur position au cours du temps.

Extensions

- Imposées ou proposées mais validée par l'enseignant
- Chaque extension doit pouvoir être activée et désactivée en utilisant le fichier de configuration config.json, plusieurs JSON peuvent être proposés
- Chaque extension devra être testée

Extensions proposées - Gravité

Cette extension consiste à modifier vos particules pour prendre en compte la gravité. Pour cela, on va simplement utiliser une constante (l'accélération de la gravité), qui sera définie au niveau du système de particules, et prendra une valeur paramétrée par config.json.

À chaque mise-à-jour d'une particule, cette valeur sera ajoutée à la vitesse selon la coordonnée y (l'ordonnée) de cette particule.

- Simple
- Pas de difficulté algorithmique

Extensions proposées - Extérieur de l'écran

Une particule qui est largement sortie de l'écran, de telle sorte qu'elle ne risque plus d'y revenir (ou en tout cas qu'on ne s'en rendra pas compte si elle ne revient pas), n'a pas de raison de continuer à être mise-à-jour et affichée.

Pour détecter cela, on pourra utiliser un nouveau champs dans le fichier de configuration, indiquant la marge en pixels en dehors de l'écran en dehors de laquelle on arrête de mettre-à-jour et d'afficher une particule. Dans la fonction Update() on ne mettra à jour que les particules qui sont encore dans la zone définie par cette marge. De plus, on n'affichera aussi que ces particules.

Pour cela, il faut modifier légèrement la fonction Draw() du fichier draw.go : il suffit d'ajouter une condition autour du code à l'intérieur de la boucle for.

- Simple
- Pas de difficulté algorithmique

Extensions proposées – Durée de vie

Il peut être intéressant, dans un système de particules, de donner une durée de vie aux particules.

On peut ainsi représenter certains phénomènes plus facilement (par exemple, on peut représenter du feu en ayant des particules bougeant vers le haut et avec une durée de vie assez courte, les faisant disparaître — idéalement en devenant de plus en plus transparentes — au bout d'un certain temps).

Cela signifie que, après un certain nombre d'appels à la fonction `Update()`, une particule arrête d'être mise-à-jour et d'être affichée.

Pour mettre en œuvre cela il faudra ajouter un champs dans le fichier de configuration, permettant de définir la durée de vie (en nombre d'appels à `Update()`) des particules.

Chaque particule comptera combien de fois elle a été mise-à-jour et, si elle dépasse la durée de vie, ne sera plus mise-à-jour ni affichée (cette deuxième partie demande de modifier légèrement la fonction `Draw()` du fichier `draw.go`, il suffit d'ajouter une condition autour du code à l'intérieur de la boucle `for`).

- Simple
- Pas de difficulté algorithmique

Extensions proposées - Variations de couleur, d'échelle, de rotation, de transparence

Proposer quelques variantes avec des rendus intéressants

- Simple
- Pas de difficulté algorithmique
- Rendu visuel

Extensions proposées – forme du générateur

Pour l'instant, votre système de particules devrait engendrer des particules de deux manières différentes : aléatoirement positionnées à l'écran ou toutes en un même point.

Il peut être intéressant de permettre d'engendrer des particules sur des formes plus complexes qu'un point. Par exemple, on pourrait les engendrer aléatoirement réparties sur un cercle.

Dans ce contexte il devient possible de choisir les vitesses initiales des particules en fonction de la forme du générateur.

Par exemple, on pourrait faire que les particules sont initialement propulsées vers l'extérieur d'un cercle.

À vous de voir comment mettre ça en œuvre et comment le paramétrer dans le fichier de configuration.

- Simple
- Pas de difficulté algorithmique
- Rendu visuel

Extensions proposées – Générateur mobile

Le générateur de votre système de particules est immobile. Parfois on souhaiterait pouvoir le déplacer (imaginez par exemple qu'on l'utilise pour représenter du feu sur une torche portée par un personnage en mouvement) mais ceci crée des complexités : les particules doivent-elles suivre le générateur ?

doivent-elles rester à leur place? le déplacement du générateur influe-t-il sur la vitesse initiale des particules ?

- Rendu visuel
- Difficulté algorithmique moyenne

Extensions proposées - Optimisation de la mémoire

Liée à l'extérieur de l'écran ou à la durée de vie

- Purement algorithmique
- Moyenne à difficile

Une particule qui n'est plus mise-à-jour ni affichée (on dira qu'elle est morte) n'a pas besoin d'être gardée en mémoire. Si on laisse le système de particules engendrer de nouvelles particules pendant longtemps, la liste qui les contient ne cesse jamais de croître et contient de plus en plus de particules mortes. Pour éviter cela, on peut supprimer les particules mortes de cette liste : à chaque fois qu'une particule devient morte, on la retire de la liste.

Extensions proposées - Optimisation de la mémoire ++

La méthode d'optimisation de la mémoire précédente a un défaut : les éléments retirés de la liste existeront toujours en mémoire, c'est simplement leur adresse qui aura été supprimée.

Ceci pourrait finir par remplir tout l'espace disponible. Pour éviter cela, le (Runtime du) langage Go met en place un ramasse-miettes (garbage collector) qui va, de temps en temps, nettoyer la mémoire pour supprimer tous les éléments dans celle-ci qui ne sont plus référencés (pour lesquels il n'existe plus de variable qui contient leur adresse dans le programme) et donc en particulier nos particules mortes qui ne sont plus référencées dans la liste.

Plus il y a de mémoire à nettoyer et plus cela prend du temps. De plus, pour des raisons de sécurité et d'intégrité des données,

le fonctionnement normal du programme doit être interrompu pendant le nettoyage de la mémoire.

On risque donc, au bout d'un moment, d'observer des ralentissements si on supprime ainsi les particules de la liste

Extensions proposées - Optimisation de la mémoire ++

- Idée recycler les particules mortes
- Purement algorithmique
- Moyenne à difficile

Extensions proposées - collisions

- Les particules dans votre systèmes n'ont pour l'instant aucune interactions entre elles. Il peut être intéressant de détecter quand deux particules se rencontrent (il y a une collision) et de réagir à ceci d'une manière ou d'une autre (en changeant la couleur, la taille, l'orientation, la vitesse, etc des particules qui se sont rencontrées).
- On peut même imaginer simuler les interactions physiques entre les particules : lors d'une rencontre elles se repoussent.
- Purement algorithmique
- Difficile

Evaluation

- Partie 1 : Système de particules de base
 - Commentaires
 - Fonctionnalités
 - newSystem
 - Update v1 et v2
 - Tests
 - Architecture
 - Package
 - Visibilités
 - Fonctions élémentaires
- Par les pairs
 - Vous évaluez et votre évaluation est évaluée
- Partie 2 : extensions
 - Fonctionnement
 - Tests
 - Code
 - Architecture
 - Performances
 - Extensions
- Oral